

Last Updated: Dec 18, 2025

Current state of MatchMiner-AI pipeline code.....	3
Limitations.....	3
“mmai” Python package.....	4
Product.....	4
Who is this for?.....	4
MVP.....	4
In scope for MVP.....	4
Out of scope for MVP.....	5
Points of feedback requested as you look through the doc.....	5
High-level pipeline workflow.....	6
Package structure.....	7
Testing.....	8
summarize_trials(...).....	9
Function signature.....	9
Example call.....	10
Tests.....	11
Other open considerations.....	11
summarize_patients(...).....	12
Function signature.....	12
Example call.....	13
Tests.....	14
Other open considerations.....	14
embed_for_matching(...).....	15
Function signature.....	15
Example call.....	16
Tests.....	17
Other open considerations.....	17
generate_candidate_matches(...).....	18
Function signature.....	18
Example call.....	20
Tests.....	20
Other open considerations.....	20
reasonable_match_check(...).....	21
Function signature.....	21
Example call.....	22
Tests.....	22
Other open considerations.....	23
exclusion_criteria_check(...).....	24
Function signature.....	24
Example call.....	25

Tests.....	25
Other open considerations.....	26
High-level pipeline functions.....	27
run_patient_centric_matching_pipeline(...)	28
Function signature.....	29
Example call.....	31
Tests.....	31
Other open considerations.....	31
run_trial_centric_matching_pipeline.....	32
build_trial_embedding_corpus(...)	33
build_patient_embedding_corpus(...)	34
query_trials_for_patients(...)	35
query_patients_for_trials(...)	36
Configuration.....	37
Design constraints.....	37
Proposed implementation.....	37
Model metadata.....	38
Appendix.....	40
Engineering to dos and considerations.....	40

Current state of MatchMiner-AI pipeline code

The MatchMiner-AI pipeline has been successfully implemented internally at DFCI to support a pilot clinical deployment, enabling clinicians to receive AI-assisted trial matches for real patients. The underlying methodology (developed by Ken Kehl) has been [published](#), and the internal production implementation (Fig 1) is well-engineered for operational use.

To support understanding of the workflow for external folks, we also have a tutorial notebook (Fig 2) that demonstrates the full end-to-end pipeline with synthetic data.

Together, these resources enable clinical use of the pipeline and support scientific communication of the approach.

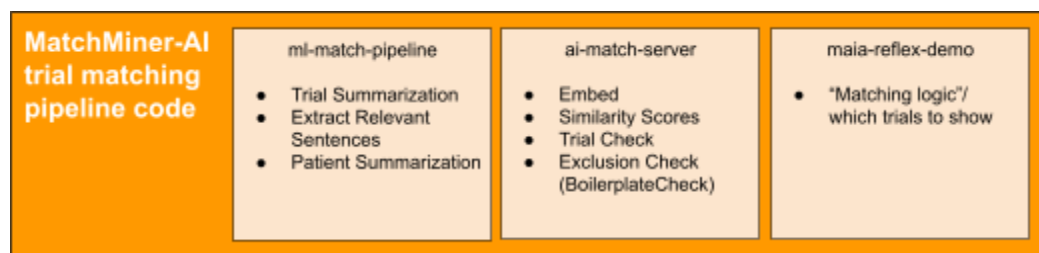


Fig 1. KSG GitLab repositories where MatchMiner-AI pipeline functions live. Adapted from [Jen](#).

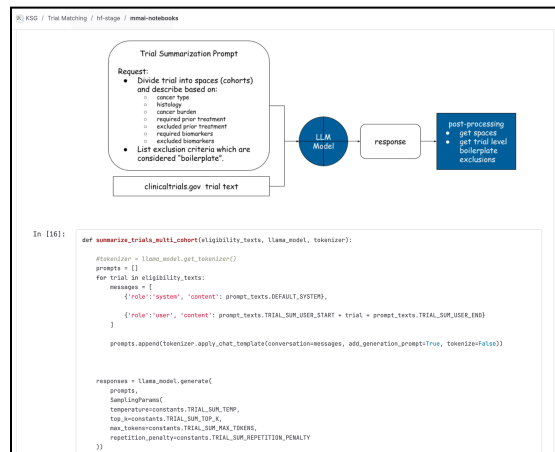


Fig 2. MatchMiner-AI pipeline code in a [tutorial notebook](#).

Limitations

While the pipeline works well operationally at DFCI, its current structure is optimized for internal deployment rather than for broader collaboration and adoption.

- Code implementing the MatchMiner-AI methodology is distributed across several private repositories and interwoven with DFCI and KSG specific infrastructure details. It would be difficult for external folks to run the pipeline.

- The tutorial notebook demonstrates the full workflow with several functions embedded, but is designed for exploration, not reproducible reuse.

“mmai” Python package

Product

An open-source Python package that provides an API for running the MatchMiner-AI pipeline. It consolidates core components into a single importable library and exposes the major pipeline phases—trial summarization, patient summarization, embedding, etc —as first-class functions, along with the ability to run the pipeline end-to-end with one function.

Who is this for?

The mmai package is designed primarily for developers and researchers who want to run, understand, or extend the MatchMiner-AI pipeline.

Audience	Purpose
Internal/ KSG	Foundation for on-campus deployment, the Reflex web app, and the tutorial notebook
External	A reproducible reference implementation of the MatchMiner-AI methodology, enabling reuse, extension, and open collaboration independent of DFCI-specific infrastructure
External	A “non-engineer” looking to just run the pipeline on their own data. Think: a Ken-like person at another academic medical center.

MVP

In scope for MVP

- Function-level public APIs for each pipeline phase
- End-to-end convenience wrapper for users who want matches without interacting with intermediate steps
- Contains all “validated” prompts, parameters, etc to run the pipeline – see [configuration](#) section
- Some [testing](#) [plans IP]
- Packaged via pip

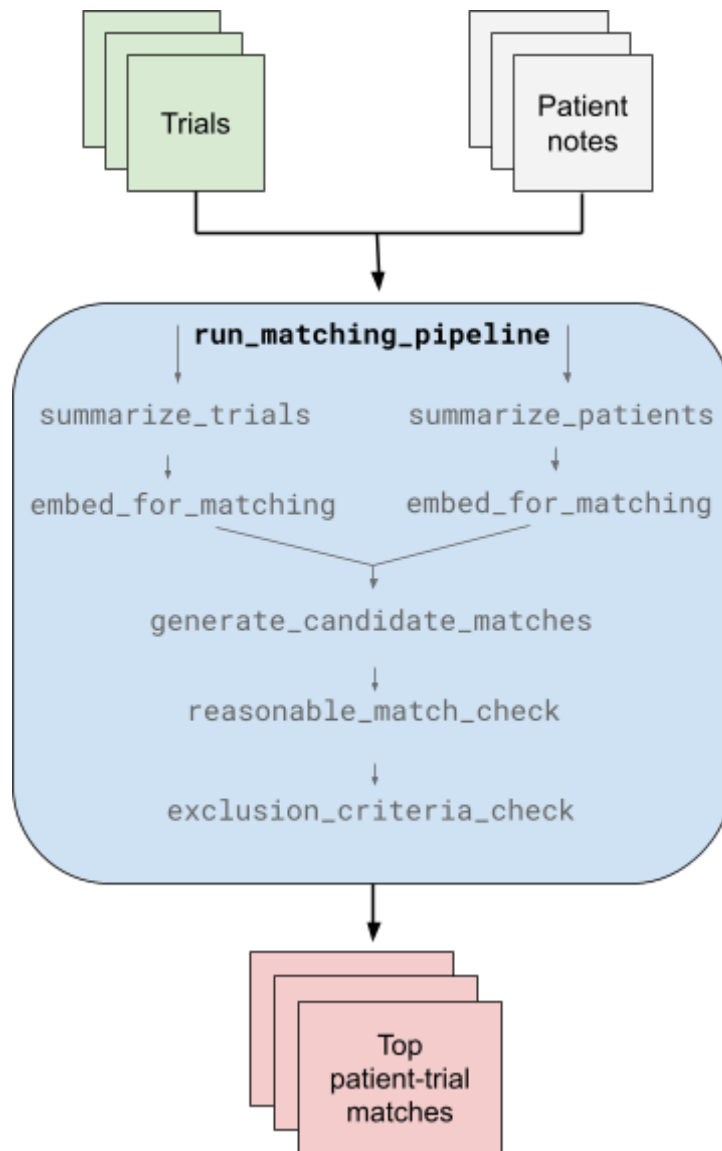
Out of scope for MVP

- Implementing steps for models hosted in server mode (vLLM web server). Although the MVP implementation will be “local mode” and allow for adding server mode later.
- Custom configurations may be used for experimentation, but only the default preset is considered supported.

Points of feedback requested as you look through the doc

- **Is the [configuration](#) philosophy reasonable?** versioned default preset as the supported path, and experimental custom configuration via optional initialization? **I’m not sure what all will break the pipeline** (e.g., changing the prompt or model) / if we shouldn't make it customizable
- **Minimum [behavior testing and summary statistics](#) we want to provide?**
 - For example, for a set of trials and synthetic notes, *just* determine if final matches change compared to a validated/baseline run? Or if the ranking changes?
- **Package name?**
- **Do these top-level functions feel like the right initial public API?**
Are the function names intuitive?
- **Thoughts on function inputs and outputs?**
 - Note that we do [plan](#) to have a ‘debug’ mode to provide more intermediate fields in the output dataframes for troubleshooting (e.g., raw LLM output)

High-level pipeline workflow



Package structure

```
mmai/
├── pyproject.toml
├── README.md
├── src/
│   └── mmai/
│       ├── __init__.py
│       ├── pipeline.py           # MMAIPipeline + end to end wrapper
│       ├── config.py            # Config schema and loading preset
│       ├── presets/
│       │   ├── default.yaml     # model names and other params
│       │   └── prompts/        # prompt files
│       │       ├── trial_summary.txt
│       │       ├── patient_summary.txt
│       │       ├── embedding.txt
│       │       └── checker_templates.txt
│       ├── trials/              # trial summarization phase
│       │   ├── __init__.py
│       │   ├── prompt_builder.py # build prompts
│       │   ├── summarize.py      # summarize trial text
│       │   └── postprocess.py    # cleaning to space and boilerplate
│       ├── patients/            # patient summarization phase
│       │   ├── __init__.py
│       │   ├── tagging.py        # relevant sentence extraction
│       │   ├── prompt_builder.py # build prompts
│       │   ├── summarize.py      # summarize "long text"
│       │   └── postprocess.py    # cleaning to summary and boilerplate
│       ├── embedding/           # embedding for matching phase
│       │   ├── __init__.py
│       │   └── embed.py          # embed text
│       ├── matching/            # matching + filtering phase
│       │   ├── __init__.py
│       │   ├── match.py          # similarity and ranking
│       │   ├── reasonable_check.py # reasonable match
│       │   └── exclusion_check.py  # boilerplate checker
│       ├── backends.py          # LLM + HF pipeline execution helpers
│       └── utils/
│           ├── __init__.py
│           └── logging.py
├── tests/
│   ├── test_pipeline.py
│   ├── test_trials.py
│   ├── test_patients.py
│   ├── test_embedding.py
│   └── test_matching.py
```

Testing

There are different types of testing we'd like to implement with this package. More concrete examples within function sections.

- Unit tests:
 - Validate that functions behave as expected
 - Examples: Input schema validation, output schema and shapes, correct behavior of flags
- Behavior/ regression tests:
 - Pipeline results compared to baseline/ gold standard results (see below test data)
 - Examples: Comparing embeddings, similarity scores, checker results, trial rankings
 - Note: This would be computationally intensive. JamesL mentioned taking advantage of free usage quotas from services like Groq
- QC reports and summary statistics:
 - Provide descriptive statistics that help monitor model and pipeline behavior over time.
 - For example, for trial summarization:
 - the number of trials missing summaries
 - number of trials with non-distinct spaces
 - ...

Test data for behavior/ regression tests will be derived from trials and synthetic notes generated for training the models (see [mmain-synthetic](#)...caution: this repo is currently still in progress).

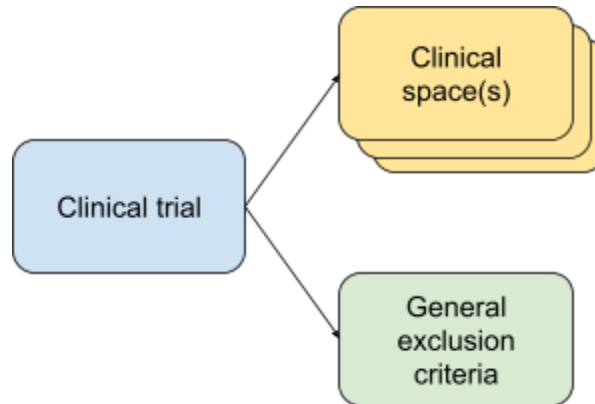
This will include subsets of:

- trial_spaces_with_positive_prompts.csv;
- trial_spaces_with_negative_prompts.csv;
- all_synthetic_notes.parquet.

summarize_trials(...)

Goal

Given clinical trial text, retrieve the clinical space(s) targeted by the trial and the general exclusion criteria that apply.



Core deliverables

- Clinical space(s)
- General exclusion criteria

Function signature

```
def summarize_trials(  
    trials: pd.DataFrame,  
) -> pd.DataFrame:  
    """  
    Summarize clinical trials into clinical spaces and general exclusion  
    criteria.
```

Parameters

trials : pd.DataFrame
 Trial-level input. One row per trial.

Expected columns

trial_id : str
 Unique trial identifier.
trial_title : str
 Human-readable trial title.
brief_summary : str
 Brief textual summary of the trial.

```

        eligibility_criteria : str
            Full eligibility criteria text for the trial.

Returns
-----
pd.DataFrame
    Clinical-space-level DataFrame. One row per clinical space per
    trial.

Columns
-----
    space_trial_id : str
        Unique identifier for a specific trial + clinical space
        combination.
    trial_id : str
        Original trial identifier (copied through from input).
    clinical_space_number : int
        Integer index of the clinical space within the trial.
    clinical_space_summary : str
        Summary of the clinical space (disease context, line of
        therapy, etc).
    general_exclusion_criteria : str
        General trial-level exclusion criteria text extracted for this
        space.

Debug Columns
(Available only if pipeline initialized with debug_mode=True)
-----
    trial_text : str
        Concatenation of trial_title + brief_summary +
        eligibility_criteria.
        This is the raw input text fed into the LLM.
    space_reasoning_and_output : str
        Raw LLM response text.
"""
...

```

Example call

```

import mmai
pipeline = mmai.MMAIPipeline()

```

```
trial_spaces_df = pipeline.summarize_trials(trials_df)
```

Tests

Data

A set of trials taken from both [trial_spaces_with_positive_prompts.csv](#) and [trial_spaces_with_negative_prompts.csv](#). (Columns 'nct_id', 'title', 'eligibility_criteria', 'trial_text'.) Corresponding synthetic patient notes should be retained for `summarize_patients()` testing.

Code

MVP = check contents (after test run, output a QC file with the following)

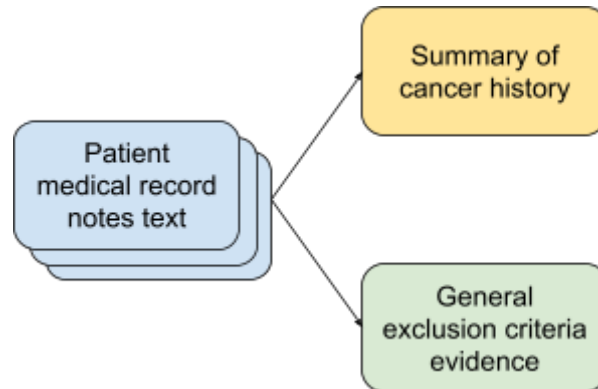
- number of trials missing summaries;
- number of spaces/trial;
- number of trials with non-distinct spaces. (Check for two or more "1" spaces or spaces with exact same text.);
- number and id of cohort spaces missing expected sections/keywords ["Age", "Sex", "Cancer type allowed", "Histology allowed", "Cancer burden allowed", "Prior treatment required", "Prior treatment excluded", "Biomarkers required", "Biomarkers excluded"];
- number and id of trials missing boilerplate exclusions (e.g. Boilerplate = 'None'); and
- number of cohort spaces with an excessive length.

Other open considerations

summarize_patients(...)

Goal

Given longitudinal patient medical record text (clinical notes, imaging reports, etc.), generate a summary of the patient's cancer history and identify evidence of conditions that correspond to common clinical trial exclusion criteria.



Core deliverables

- Summary of the patient's cancer history
- Summary of conditions that correspond to common clinical trial exclusion criteria

Function signature

```
def summarize_patients(  
    notes: pd.DataFrame,  
) -> pd.DataFrame:  
    """  
    Summarize longitudinal patient notes into a cancer history summary and  
    evidence related to general exclusion criteria.  
  
    Parameters  
    -----  
    notes : pd.DataFrame  
        Note-level input. One row per note.  
  
    Expected columns  
    -----  
    patient_id : str  
        Unique patient identifier.  
    note_text : str
```

```

        Full note text.
    note_type: str
        Type of note (clinical_note, pathology_report, etc.).
    note_date : str or datetime
        Date of the note.

Returns
-----
pd.DataFrame
    Patient-level summary DataFrame. One row per patient.

Columns
-----
patient_id : str
    Unique patient identifier.
cancer_history_summary : str
    Summary of the patient's cancer history.
general_exclusion_criteria_evidence : str
    Summary of conditions / findings that correspond to common
    clinical trial exclusion criteria.

Debug Columns
(Available only if pipeline initialized with debug_mode=True)
-----
patient_long_text : str
    Concatenated set of tagger-selected (relevant) patient note
text
    used as the input context for the LLM.
original_patient_summary : str
    Raw LLM response generated from the patient long text.
cleaned_patient_summary : str
    LLM response with reasoning text removed.
"""
...

```

Example call

```
patient_summary_df = pipeline.summarize_patients(notes_df)
```

Tests

Data

A set of synthetic patient notes extracted from [all_synthetic_notes.parquet](#). The set of notes chosen should correspond to synthetic patients generated for the trials selected for `summarize_trials()`.

Code

MVP = check contents (after test run, output a QC file with the following)

- number of patient with no tagged notes (intermediate step output check);
- number of patients with no summary generated;
- number of patients for which patient summary == boilerplate exclusions. (This indicates that the boilerplate section was not properly extracted.);
- number and id of patient summaries missing expected sections/keywords, enumerated by each missing keyword ['Cancer type', 'Histology', 'Current extent', 'Biomarkers', 'Treatment history']
- number and id of patient summaries with an excessive length;

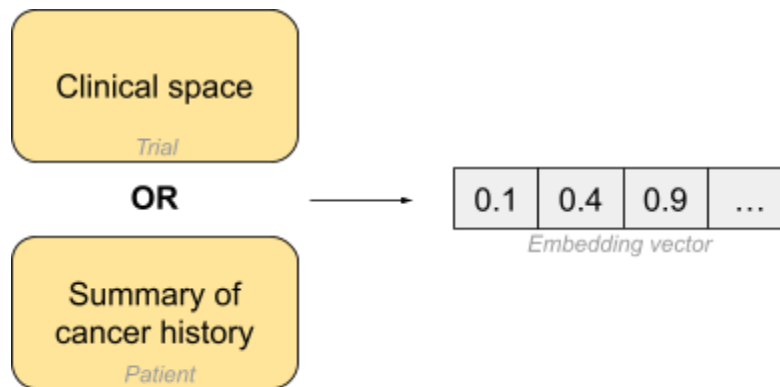
Other open considerations

- Would we rather this be separate steps like done in prod – extracting relevant sentences and the summarizing patient? I put these steps together because I like the symmetry and I couldn't imagine a situation where these steps wouldn't be performed together...
- Do we want to include any functionality/documentation about cleaning/preparing the notes themselves even before extracting relevant sentences?

embed_for_matching(...)

Goal

Transform patient or trial summaries (from `summarize_*` functions) into embedding vectors in a shared semantic space where distance reflects similarity.



Core deliverables

Embedding vector

Function signature

```
def embed_for_matching(  
    df: pd.DataFrame,  
    *,  
    entity_type: Literal["patient", "trial"],  
) -> pd.DataFrame:  
    """  
    Convert trial or patient summaries into embedding vectors for semantic  
    matching.
```

Parameters

df : pd.DataFrame
Input summaries to embed.

For **entity_type="patient"**
One row per patient.

Expected columns

```

        cancer_history_summary : str
            Summary text to embed for each patient.

    For entity_type="trial"
        One row per clinical space.

    Expected columns
    -----
        clinical_space_summary : str
            Summary text to embed for each clinical space.

    entity_type : {"patient", "trial"}
        Controls which summary column is used as the text to embed.

    Returns
    -----
    pd.DataFrame
        Same DataFrame with an added embedding column.

    Columns (in addition to existing)
    -----
    embedding : array-like
        Vector representation of the summary text in a shared semantic
space.
    """
    ...

```

Example call

```

embedded_spaces_df = pipeline.embed_for_matching(
    trial_spaces_df,
    entity_type="trial",
)

embedded_patients_df = pipeline.embed_for_matching(
    patient_summary_df,
    entity_type="patient",
)

```


Tests

Data

Input: Subset of [patient_summaries_and_their_spaces.parquet](#), using the same trial spaces and patients used for `summarize_trials()` and `summarize_patients()`. (Note: link for `patient_summaries_and_their_spaces.parquet` is to a repository which is still under construction.)

Output for comparison: Embedding outputs for the spaces and patients when run in our hands (`embedding_gold_standard`).

Code

MVP = new embeddings vs. gold standard embeddings.

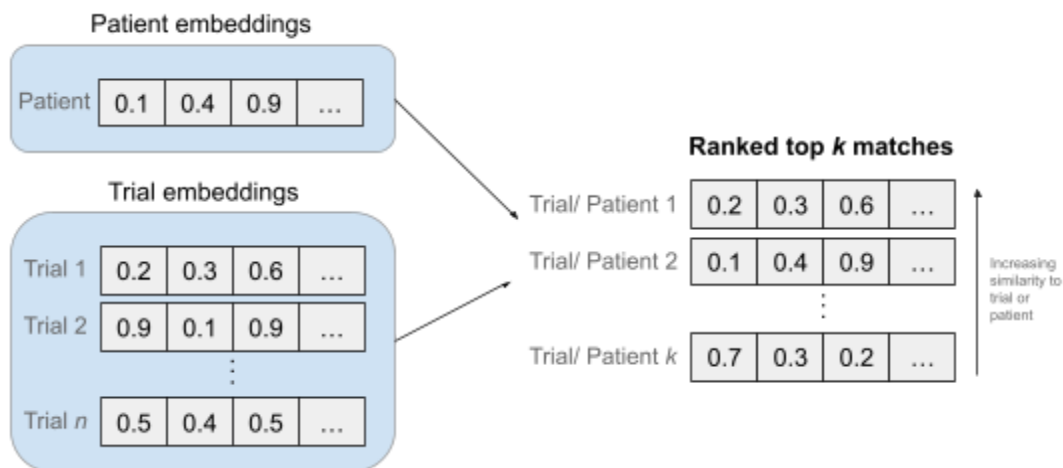
- Compare with embedding gold standard. Output a variance metric (e.g. similarity scores):
 - Similarity scores comparing new patient embeddings to patient `embedding_gold_standard`; and
 - Similarity scores comparing new trial space embeddings to trial `embedding_gold_standard`.

Other open considerations

generate_candidate_matches(...)

Goal

Generate candidate patient–trial match pairs by computing similarity between patient and trial-space embeddings, ranking trial spaces for each patient or patients for each trial space, and returning the top- k results.



Core deliverables

- Patient-centric usage: query_df = patients, corpus_df = trial_spaces → ranked top-k trial spaces per patient
- Trial-centric usage: query_df = trial_spaces, corpus_df = patients → ranked top-k patients per trial space.

Function signature

```
def generate_candidate_matches(  
    query_df: pd.DataFrame,  
    corpus_df: pd.DataFrame,  
    *,  
    k: int = 20,  
    query_id_col: str,  
    corpus_id_col: str,  
) -> pd.DataFrame:  
    """
```

Generate top-k candidate matches by ranking corpus items for each query item using similarity computed from embedding vectors.

Parameters

```

query_df : pd.DataFrame
    Query-side DataFrame with embeddings.
    Must contain:
        - <query_id_col>: str
        - embedding : array-like

    Additional columns will be preserved and returned unchanged for
    contextual use.

corpus_df : pd.DataFrame
    Corpus-side DataFrame with embeddings.
    Must contain:
        - <corpus_id_col>: str
        - embedding : array-like

    Additional columns will be preserved and returned unchanged.

k : int, default 20
    Number of top matches to return per query entity (top-k corpus
    items per query).

query_id_col : str
    Column name for the query entity identifier (e.g., "patient_id" or
    "space_trial_id").

corpus_id_col : str
    Column name for the corpus entity identifier (e.g.,
    "space_trial_id" or "patient_id").

Returns
-----
pd.DataFrame
    Ranked top-k match pairs including:
        - <query_id_col>
        - <corpus_id_col>
        - similarity_score : float

    plus any additional columns from the input DataFrames for context.
"""
...

```

Example call

```
# Patient-centric: top k unique trials per patient (best space chosen per trial)
candidate_matches_patient = pipeline.generate_candidate_matches(
    query_df=embedded_patients_df,
    corpus_df=embedded_spaces_df,
    k=20,
    query_id_col="patient_id",
    corpus_id_col="space_trial_id",
)

# Trial-centric: top k patients per trial space
candidate_matches_trial = pipeline.generate_candidate_matches(
    query_df=embedded_spaces_df,
    corpus_df=embedded_patients_df,
    k=20,
)
```

Tests

Data

Input: embedding_gold_standard.

Output for comparison: Similarity scores when embedding_gold_standard is run in our hands (similarity_score_gold_standard).

Code

MVP = new similarity scores vs similarity_score_gold_standard.

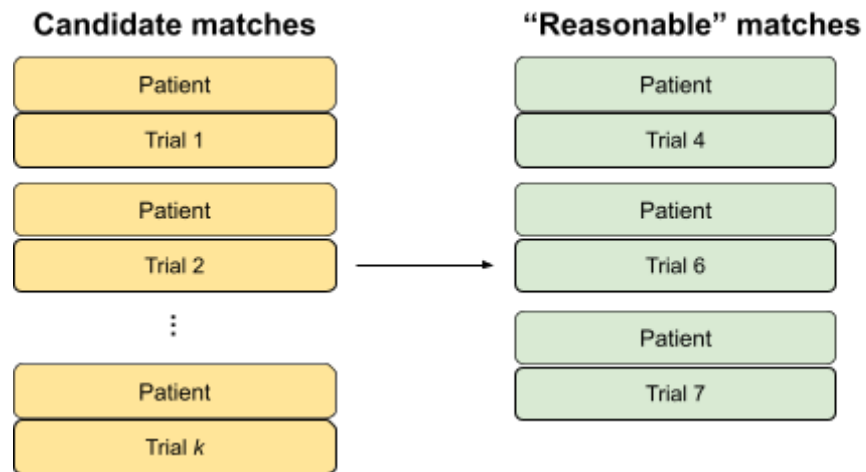
- Generate flag if these are not equal.
- If not equal:
 - report degree of variance in scores; and
 - compare rankings.

Other open considerations

reasonable_match_check(...)

Goal

Given top k candidate patient-trial pairs, evaluate whether each trial is a reasonable clinical consideration for the patient based on the patient summary and the trial clinical-space summary.



Core deliverables

Determination of which candidate patient-trial matches are reasonable matches.

Function signature

```
def reasonable_match_check(
    candidate_matches: pd.DataFrame,
    *,
    filter_unreasonable: bool = True,
) -> pd.DataFrame:
    """
    Evaluate whether each candidate patient-trial pair represents a
    clinically reasonable match.

    Parameters
    -----
    candidate_matches : pd.DataFrame
        Output from generate_candidate_matches(...).

    Expected columns
    -----
    cancer_history_summary : str
        Patient summary text.
```

```

        clinical_space_summary : str
            Trial clinical-space summary text.

    filter_unreasonable : bool, default True
        If True, only return rows where ``reasonable_match`` evaluates to
        True.

    Returns
    -----
    pd.DataFrame
        Same DataFrame with the following additional columns:

    Columns (added)
    -----
    reasonable_match_score: float
        Model-generated confidence score that the match is clinically
        reasonable match.
    reasonable_match : bool
        Whether the candidate patient-trial pair is considered
        a clinically reasonable match.
    """
    ...

```

Example call

```

reasonable_df = pipeline.reasonable_match_check(
    candidate_matches_df,
    filter_unreasonable=True,    # Filter unreasonable matches
)

```

Tests

Data

Input:

Top 20 matches from `generate_candidate_matches()` (similarity_gold_standard)

Output for comparison:

`reasonable_trial_check()` outputs in our hands (`reasonable_match_gold_standard`). In creating this gold standard, we may wish to check the originating data (from `trial_spaces_with_positive_prompts` and `trial_spaces_with_negative_prompts` and/or `patient_summaries_and_their_spaces.parquet`) to see if eligibility agrees.

Code

For test run set filter_unreasonable to False.

MVP

- Compare label (boolean) for each patient-trial space pair to reasonable_match_gold_standard.
- Output report with agreement and disagreement stats (and confidence scores).

Extension

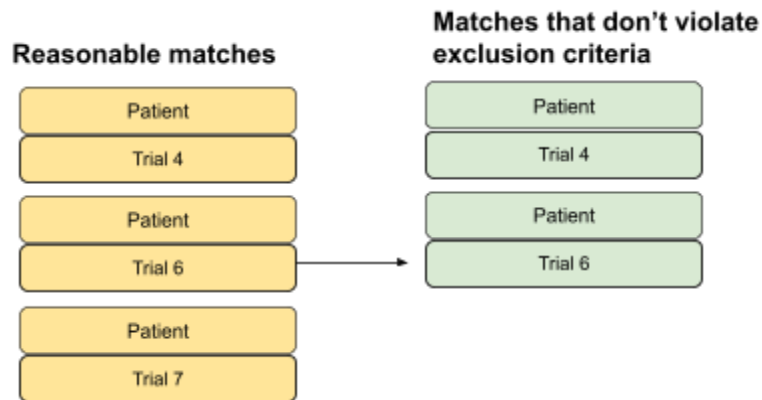
- report degree of variance in confidence scores for each patient-trial space pair.

Other open considerations

exclusion_criteria_check(...)

Goal

Evaluate candidate patient–trial matches that passed the reasonability check to determine whether the patient meets any of the trial's exclusion criteria.



Core deliverables

Determination of which candidate patient-trial matches violate or don't violate exclusion criteria.

Function signature

```
def exclusion_criteria_check(  
    matches: pd.DataFrame,  
    *,  
    filter_excluded: bool = False,  
) -> pd.DataFrame:  
    """  
    Evaluate whether patients meet any exclusion criteria for their  
    candidate trial match.  
  
    Parameters  
    -----  
    matches : pd.DataFrame  
        Output from reasonable_match_check(...).  
  
    Expected columns  
    -----  
    general_exclusion_criteria : str  
        Trial-level exclusion criteria text for the clinical space.  
    general_exclusion_criteria_evidence : str
```


Patient-level evidence related to exclusion criteria.

filter_excluded : bool, default False

If True, return only matches where exclusion criteria are not violated.

Returns

pd.DataFrame

Same DataFrame with the following additional columns:

Columns (added)

exclusion_score: float

Model-generated confidence score that the patient does not violate the trial's exclusion criteria.

exclusion_criteria_pass : bool

True if the patient does not appear to violate the trial's exclusion criteria; False otherwise.

"""

...

Example call

```
exclusion_checked_df = pipeline.exclusion_criteria_check(  
    reasonable_df,  
    filter_excluded=False,    # keep exclusion criteria violations  
)
```

Tests

Data

Input: filtered output from `reasonable_match_check()` (filtered `reasonable_match_gold_standard`).

Output for comparison: `exclusion_criteria_check()` run in our hands. (`exclusion_check_gold_standard`).

Code

For test run, set `filter_excluded` to False.
MVP

- Compare label (boolean) for each patient-trial space pair to `exclusion_check_gold_standard`.
- Output report with agreement and disagreement stats (and confidence scores).

Extension

- report degree of variance in confidence scores for each patient-trial space pair.

Other open considerations

High-level pipeline functions

These functions orchestrate multiple pipeline phases.

run_patient_centric_matching_pipeline(...)

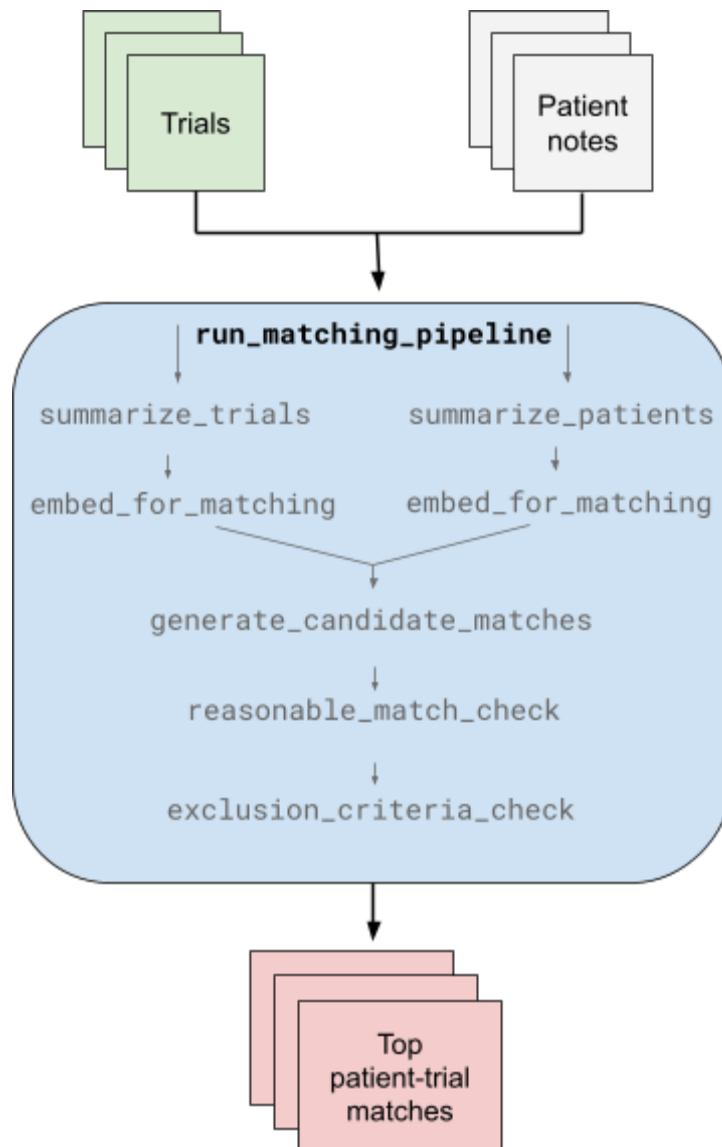
Intended use case

Use this function when you want to run the entire MatchMiner-AI pipeline end to end from raw trial metadata and raw patient notes, returning top-k trials per patient.

Goal

Given trial metadata and patient notes, run the pipeline end to end to generate scored patient-trial match pairs, returning the top k unique trials per patient (patient-centric view).

Internally, this calls `summarize_trials`, `summarize_patients`, `embed_for_matching`, `generate_candidate_matches`, `reasonable_match_check`, and `exclusion_criteria_check`.



Function signature

```
def run_patient_centric_matching_pipeline(
    trials: pd.DataFrame,
    notes: pd.DataFrame,
    *,
    k: int = 20,
    filter_unreasonable: bool = True,
    filter_excluded: bool = False,
) -> pd.DataFrame:
    """
    Run the end-to-end MatchMiner-AI patient-centric matching pipeline,
    from
    summarization through embedding, candidate generation, reasonability
    evaluation, and exclusion checks.

    Parameters
    -----
    trials : pd.DataFrame
        Trial-level input with one row per trial.

        Expected columns
        -----
        trial_id : str
            Unique trial identifier.
        trial_title : str
            Trial title.
        brief_summary : str
            Brief textual summary of the trial.
        eligibility_criteria : str
            Full eligibility criteria text (inclusion and exclusion).

    notes : pd.DataFrame
        Patient note-level input with one row per note.

        Expected columns
        -----
        patient_id : str
            Unique patient identifier.
        note_text : str
            Full note text.
        note_date : str or datetime
            Date of the note.
```

note_type : str
Type of note (clinical_note, pathology_report, etc.).

k : int, default 20
Number of top unique trials to return per patient.

filter_unreasonable : bool, default True
If True, filter out matches that are not clinically reasonable
(using reasonable_match_check).

filter_excluded : bool, default False
If True, filter out matches where exclusion criteria are violated
(using exclusion_criteria_check).

Returns

pd.DataFrame
Patient-trial match results with columns derived from intermediate stages.

Output shape

One row per (patient_id, trial_id), with at most k trials per patient.

When a trial has multiple clinical spaces, the most similar space is selected for each (patient_id, trial_id) pair.

Columns (typical)

patient_id : str
space_trial_id : str
trial_id : str
trial_title : str
brief_summary : str
eligibility_criteria : str
clinical_space_number : int
clinical_space_summary : str
general_exclusion_criteria : str
reasonable_match : bool
reasonable_match_score : float
exclusion_criteria_pass : bool
exclusion_score : float

```
....
```

```
...
```

Example call

```
results_df = pipeline.run_patient_centric_matching_pipeline(  
    trials=trials_df,  
    notes=notes_df,  
    k=20,  
    filter_unreasonable=True,  
    filter_excluded=False,  
)
```

Tests

Data

Same inputs as those used for each subfunction.

Code

Run each intermediate step set of tests and generate a final report.

Other open considerations

`run_trial-centric_matching_pipeline`

Not defined yet in MVP but to be included.

`build_trial_embedding_corpus(...)`

Intended use case

Use this function to pre-compute and store trial (clinical space) embeddings ahead of time for repeated downstream matching against many patients in an online or production setting.

How this function is composed

Wraps `summarize_trials(...)` and `embed_for_matching(entity_type="trial")` to produce a clinical-space-level DataFrame that includes both the trial summarization outputs and the corresponding embedding vectors for use in on-the-fly trial matching (for example, via `query_trials_for_patients(...)`).

`build_patient_embedding_corpus(...)`

Intended use case

Use this function to pre-compute and store patient embeddings ahead of time for repeated downstream matching against many trials in an online or production setting.

How this function is composed

Wraps `summarize_patients(...)` and `embed_for_matching(entity_type="patient")` to produce a patient-level DataFrame that includes both the patient summarization outputs and the corresponding embedding vectors for use in on-the-fly patient matching (for example, via `query_patients_for_trials(...)`).

query_trials_for_patients(...)

Intended use case

Use this function for patient-centric, online matching when trial embeddings have already been pre-computed. Given one or more new patients, the function returns the top-k matching trials per patient.

This is the primary entry point for scenarios where trial representations are treated as a static corpus and patients are evaluated incrementally (e.g., at query time).

Inputs

- **Pre-computed trial embeddings**
 - A DataFrame containing trial embeddings generated upstream (e.g., via `summarize_trials` → `embed_for_matching`).
- **Raw patient notes**
 - A DataFrame containing one or more patients with associated clinical notes.

How this function is composed

For the new patient(s), this function runs `summarize_patients(...)` followed by `embed_for_matching(entity_type="patient")`. The resulting patient embeddings and pre-computed trial embeddings are used as inputs to `generate_candidate_matches(...)`. The resulting matches are passed to the checkers to evaluate whether each patient–trial pair is a reasonable consideration and whether any exclusion criteria are violated.

query_patients_for_trials(...)

Intended use case

Use this function in a trial-centric, online matching setting where patient embeddings have already been pre-computed, and you want to find the top-k matching patients for one or more new trials.

This is the primary entry point for scenarios where patient representations are treated as a static corpus and trials are evaluated incrementally (e.g., at query time).

Inputs

- **Pre-computed patient embeddings**
 - A DataFrame containing patient embeddings generated upstream (e.g., via `summarize_patients` → `embed_for_matching`).
- **Raw patient notes**
 - A DataFrame containing trial metadata.

How this function is composed

For the new trial(s), this function runs `summarize_trials(...)` followed by `embed_for_matching(entity_type="trial")`. The resulting trial embeddings and the pre-computed patient embeddings are then used in `generate_candidate_matches(...)`. The resulting matches are passed to the checkers to evaluate whether each patient–trial pair is a reasonable consideration and whether any exclusion criteria are violated.

Configuration

Design constraints

- Pipeline components are tightly interdependent; downstream steps rely on specific formats and behaviors produced upstream (e.g., boilerplate checker assumes summarization step had a prompt that included getting exclusion criteria information; embedding step only produces meaningful embeddings if summaries were produced in a specific way)
- To preserve scientific and clinical reproducibility, each release ships with a versioned default preset that defines the “validated” configuration (models, prompts, parameters).
- Custom configurations may be used for experimentation, but only the default preset is considered supported.

Proposed implementation

The `mmai` package will use a pipeline initialization model to manage configuration. Initializing without arguments loads the supported default preset.

```
import mmai
pipeline = mmai.MMAIPipeline() # uses the default preset
```

For research and experimentation, users may clone and modify the preset and pass it at initialization (e.g., paths to new prompt files, max model length value, etc):

```
config = mmai.load_preset("default")
config.trial_summarization_model = "different_model"
pipeline = mmai.MMAIPipeline(config=config) # experimental
```

This design provides a stable, reproducible default while offering a structured path for experimentation. It also sets the foundation for future extensibility (e.g., backend selection, named presets) without complicating the v1 API.

Example options currently configurable in production

```
- debug_mode (Parameter to add extra fields to the DF)
- Backend (local vs server)
# vllm.LLM initialization
- max_model_len
- tensor_parallel_size
- gpu_memory_utilization
```

```

# llm.generate sampling params
- temperature
- Top_k (or top_p)
- max_tokens
- repetition_penalty
# HF pipeline
- Consider HF caching
- device
- Other options technically configurable in loading bert model for
  checker function but doesn't look like we use them
- TrialChecker BERT classification model max sequence length
- BoilerplateChecker BERT classification model max sequence length
# prompts/ templates
- Patient primer and question
- Trial primer and question
- Checker templates
- Checker prompt and question for LLM based reasonable consideration
  checking
- Boilerplate checker prompt and question for LLM based reasonable
  consideration checking
- Embed prompt
- Embed max sequence length
# other
- model names for every step
- patient_longnotes_max_tokens
- tagger pipeline threshold cut offs

```

Model metadata

In production, model metadata as well as different parameters used are added as dataframe fields for each step.

For example, during trial summarization, the fields “model_name” and “model_info” are added to the resulting dataframe. Model info, I think, is a JSON formatted string with

- vLLM initialization parameters (Max_model_len, Tensor_parallel_size, Gpu_memory_utilization)
- Subset of variables from the static.py file
 - Temperature, top_k, max_tokens, repetition penalty (both trial and patient defined), text_token_threshold, patient_text_md5

- Model info (model_sha, created_at, last_modified)

For the package, we propose returning an accompanying metadata JSON alongside primary DataFrame outputs. Metadata return will be controlled by a configuration flag (or function argument).

This metadata JSON will contain:

- Configuration snapshot: A snapshot of the pipeline configuration used for the call.
- Runtime model information: Lightweight identifiers for the models that were instantiated during execution (e.g. model sha, created_at, last_modified). If the function uses multiple models (e.g. summarize_patients), model information for both will be recorded within the metadata JSON

Users who wish to embed provenance directly into the DataFrame will have to do that themselves - selecting certain metadata fields and copying it into the DataFrame.

Appendix

Engineering to dos and considerations

- Where should the different types of testing live in the package? Should these all live in the package?
- How should we think about versioning?
- Implement embedding in batching mode

Documentation

- How to get to the trial and patient DF to feed in
 - Consider a function to pull in trial data from clinicaltrials.gov
- Guidance on order of steps of pipeline being important due to how things were trained (e.g. checker only on top matches)

Future dev

- Trial centric
 - Build out `run_trial_centric_matching_pipeline`
- Remotely hosted models
- LLM-based checkers
 - We want to keep the input/output contracts stable, but if in the future for the checker implementation we want to allow the LLM-based ones which dont produce a score.... How should we do this